

Department of Electrical and Computer Engineering
COMSATS University Islamabad, Attock Campus

Lab Rubrics Form	
Lab Name	Array Manipulation and Sorting in C Programming
Lab Number	10
Submitted by:	
Student Names	Registration #
Sumaira Safeer	FA22-BCE-019

Rubrics		Marks		
		1	2	3
PLO-5 Modern Tools Usage	Ability to execute the experiment utilizing hardware, software tools, or a combination of both.			
PLO-10 Communication Skills	Present lab activities effectively through detailed lab reports, oral examinations, and engaging presentations.			

Lab 10: Array Manipulation and Sorting in C Programming

Objective

To understand the use of one-dimensional arrays, implement searching and sorting algorithms, and manipulate array elements using user-defined functions in C programming.

Software Requirements

- Code::Blocks IDE / Dev-C++
- GCC Compiler
- Windows Operating System

Experiments Performed

In-Lab Tasks

1. Search for a number in an array and count the number of its occurrences.
2. Pass an array to a function and modify each element by multiplying it by three.

Post-Lab Task

1. Implement the Bubble Sort algorithm to sort five numbers in ascending order and perform its flowchart and dry run.

Learning Outcomes

After completing this lab, the student will be able to:

- Declare and manipulate one-dimensional arrays.
- Search for elements within an array.
- Count the frequency of an element.
- Pass arrays to user-defined functions.
- Modify array elements through functions.
- Implement the Bubble Sort algorithm for sorting data.
- Understand basic algorithm design through flowcharts and dry runs.

In-Lab Tasks:

Task 1:

Ten numbers are entered from the keyboard into an array. The number to be searched is entered through the keyboard by the user. Write a program to find if the number to be searched is present in the array and if it is present, display the number of times it appears in the array.

Code:

```

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int array[10];
    int num,size=0;
    for (int i=0;i<10;i++)
    {
        printf("Enter num %d=",i);
        scanf("%d", &array[i]);
    }
    printf("Enter the num to search=");
    scanf("%d", &num);

    for ( int i=0;i<10;i++)
    {
        if (array[i] == num)
            printf("search is found at location [%d]\n",i);
    }
    size++;
}

if (size> 0)
{
    printf("The number %d is present in the array.\n", num);

    printf("The num %d appears %d times in the array.\n", num,size);
}
else
{
    printf("The num %d is not found in the array.\n", num);
}
return 0;
}

```

Output:

```

"E:\PF my projects\2nd_lab\quiz 1\quiz no 1\quize 1\quize 1\lab 28 mrch\TEST\bin\Debug
Enter num 0=2
Enter num 1=3
Enter num 2=4
Enter num 3=5
Enter num 4=6
Enter num 5=7
Enter num 6=7
Enter num 7=7
Enter num 8=8
Enter num 9=8
Enter the num to search=7
search is found at location [5]
search is found at location [6]
search is found at location [7]
The number 7 is present in the array.
The num 7 appears 10 times in the array.

Process returned 0 (0x0)   execution time : 12.475 s
Press any key to continue.

```

Task 2:

Write a program which performs the following tasks:

- Initialize an integer array of 10 elements in main ()
- Pass the entire array to a function modify ()
- In modify () multiply each element of array by 3
- Return the control to main () and print the new array elements in main ()

Code:

```

#include <stdio.h>
int main()
{
    int arr[10] = {5, 10, 15, 20, 25, 30, 35, 40, 45, 50};
    int i;
    modify(arr, 10);
    printf("New array elements: ");
    for(i=0; i<10; i++)
    {
        printf("%d ", arr[i]);
    }
    return 0;
}
void modify(int arr[], int size)
{
    int i;
    for(i=0; i<size; i++)

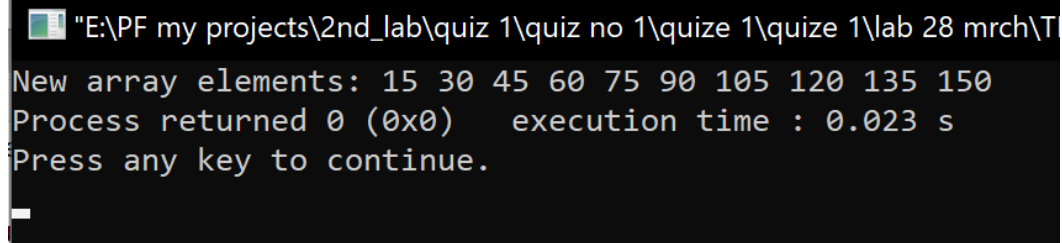
```

```

    {
        arr[i] *= 3;
    }
}

```

Output:



```

"E:\PF my projects\2nd_lab\quiz 1\quiz no 1\quize 1\quize 1\lab 28 mrch\T
New array elements: 15 30 45 60 75 90 105 120 135 150
Process returned 0 (0x0)   execution time : 0.023 s
Press any key to continue.

```

Post lab Tasks:

Task 1:

Implement the Bubble Sort algorithm on a set of 5 numbers.

- Make flow chart for this algorithm.
- Do the dry run for the first iteration of outer loop.

Code:

```

#include <stdio.h>
int main()
{
    int numbers[5] = {5, 3, 1, 4, 2};
    int i;
    printf("Before sorting: ");
    for (i = 0; i < 5; i++)
    {
        printf("%d ", numbers[i]);
    }
    numtoSort(numbers, 5);
    printf("\nAfter sorting: ");
    for (i = 0; i < 5; i++)
    {
        printf("%d ", numbers[i]);
    }
    return 0;
}

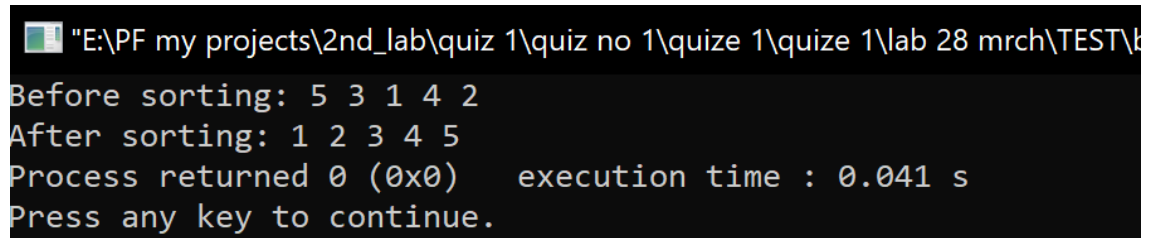
```

```

}
void numtoSort(int arr[], int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

```

Output:



```

"E:\PF my projects\2nd_lab\quiz 1\quiz no 1\quize 1\quize 1\lab 28 mrch\TEST\k
Before sorting: 5 3 1 4 2
After sorting: 1 2 3 4 5
Process returned 0 (0x0)   execution time : 0.041 s
Press any key to continue.

```